

Algorithms & Data Structures

Exam & Viva Study Notes

A concise but detailed guide covering what each topic is, why it is used, the most optimized standard complexity, and the reason behind the complexity.

1. LCS — Longest Common Subsequence

What is it? LCS finds the longest sequence common to two sequences while preserving relative order. Elements may be skipped, but their order cannot change.

Example: A = ABCDGH, B = AEDFHR → one LCS is ADH.

Why use it? Sequence comparison, text/file similarity, version comparison, DNA/sequence analysis, and dynamic programming problems.

Core idea: If $A[i] == B[j]$, take $1 + \text{LCS}(i-1, j-1)$. Otherwise take $\max(\text{LCS}(i-1, j), \text{LCS}(i, j-1))$.

Time: $O(nm)$ with standard DP because there are $n \times m$ states and each state takes $O(1)$.

Space: $O(nm)$ for the full table; $O(\min(n, m))$ if only the length is needed using rolling rows.

2. LIS — Longest Increasing Subsequence

What is it? LIS finds the longest subsequence whose values are strictly increasing.

Example: {10, 9, 2, 5, 3, 7, 101, 18} → length 4, e.g. 2, 3, 7, 101.

Why use it? To find increasing patterns in sequences; common in optimization, scheduling, trend analysis, and DP problems.

Optimized idea: Maintain a tails array and use binary search to place each element at its proper position.

Time: $O(n \log n)$ because n elements each require $O(\log n)$ binary search.

Space: $O(n)$.

3. Connected Components

What is it? A connected component is a maximal group of vertices in an undirected graph where every vertex is reachable from every other vertex in that group.

Example: 1–2–3, 4–5, and isolated 6 → three components.

Why use it? To count or identify separate groups in networks, graphs, social connections, and grid/graph problems.

Approach: DFS or BFS. Start from every unvisited vertex; one traversal visits one complete component.

Time: $O(V+E)$ with an adjacency list because every vertex and edge is processed only a constant number of times.

Space: $O(V+E)$ including graph storage; auxiliary traversal space is $O(V)$.

4. Floyd-Warshall

What is it? Floyd-Warshall is an all-pairs shortest path algorithm. It finds shortest distances between every pair of vertices.

Why use it? When shortest paths are needed for all vertex pairs and the number of vertices is manageable.

Core idea: For each possible intermediate vertex k , check whether $i \rightarrow k \rightarrow j$ is shorter: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.

Time: $O(V^3)$ because of three nested loops over k , i , and j .

Space: $O(V^2)$ for the distance matrix.

Note: Negative edges are allowed, but a negative cycle makes shortest-path values undefined.

5. Segment Tree

What is it? A Segment Tree is a tree-based data structure for efficient range queries with updates.

Common uses: Range Sum, Range Minimum, Range Maximum, and similar associative range operations.

Why use it? If an array has many queries and updates, scanning a range every time may be too slow. Segment Tree reduces standard query/update operations to logarithmic time.

Time: Build $O(n)$; point update $O(\log n)$; standard range query $O(\log n)$.

Why? The tree height is $O(\log n)$, so only relevant segments are visited.

Space: $O(n)$ (commonly an array around $4n$ is allocated).

6. Maximum Sum Subarray — Kadane's Algorithm

What is it? It finds the maximum sum among all contiguous subarrays.

Example: $\{-2, 1, -3, 4, -1, 2, 1, -5, 4\} \rightarrow \{4, -1, 2, 1\}$, sum = 6.

Why use it? It solves maximum contiguous subarray sum efficiently and is a classic one-pass optimization/DP technique.

Core idea: At each element decide whether to start a new subarray or extend the current one: $\text{current} = \max(\text{arr}[i], \text{current} + \text{arr}[i])$.

Time: $O(n)$ because every element is processed once.

Space: $O(1)$ because only a few variables are needed.

7. DSU Cycle Detection

What is it? DSU (Disjoint Set Union), or Union-Find, maintains disjoint sets using Find and Union operations.

Why use it? For cycle detection in an undirected graph. Before adding edge (u, v) , check $\text{Find}(u)$ and $\text{Find}(v)$. If they are already equal, u and v are in the same component, so adding the edge creates a cycle.

Optimization: Path compression + Union by Rank/Size.

Time: $O(E \alpha(V))$ for E edges, where α is the inverse Ackermann function; practically almost $O(E)$.

Space: $O(V)$ for parent and rank/size arrays.

8. MST Using DSU and Kruskal

What is it? A Minimum Spanning Tree is a spanning tree of a connected, weighted, undirected graph with minimum total edge weight. It has $V-1$ edges and no cycle.

How Kruskal works: 1) Sort edges by increasing weight. 2) Process from smallest to largest. 3) Add an edge only if it does not create a cycle. 4) Use DSU to test connectivity.

Why use it? To construct an MST efficiently, especially when the graph is naturally represented as an edge list.

Time: $O(E \log E)$. Sorting dominates; DSU operations are nearly linear.

Space: $O(V+E)$ for DSU and the edge list.

9. 0/1 Knapsack

What is it? Given n items with weights and values and a bag capacity W , maximize total value. Each item is either taken once or not taken at all.

Why use it? Selection/resource-allocation problems where every item has a take-or-skip decision.

DP state: $dp[i][w]$ = maximum value using the first i items with capacity w . For each item, choose the better of not taking it and taking it if it fits.

Time: $O(nW)$ because there are $n \times W$ states and each takes $O(1)$.

Space: $O(nW)$ for 2D DP; optimized 1D DP reduces it to $O(W)$.

Important: $O(nW)$ is pseudo-polynomial because W is a numeric capacity value.

10. Fractional Knapsack

What is it? Similar to knapsack, but an item may be taken partially. For example, part of a 10 kg item can be selected.

Why use it? When items are divisible, such as liquid, grain, or raw material.

Greedy strategy: Compute value/weight for every item, sort by decreasing ratio, take the highest-ratio items first, and take a fraction of the last item if needed.

Why greedy works: Every unit of capacity is assigned to the available item with the highest value per unit weight.

Time: $O(n \log n)$ due to sorting; the final scan is $O(n)$.

Space: $O(n)$ when item/ratio information is stored.

11. N-Queen — Backtracking

What is it? Place N queens on an $N \times N$ chessboard so that no two queens attack each other. Queens cannot share a row, column, or diagonal.

Why use backtracking? We try a valid position, move to the next row, and if the choice later leads to failure, undo it and try another position.

Typical approach: Place one queen per row and check the column and both diagonals before placing.

Time: Worst-case is commonly expressed as approximately $O(N!)$ for standard backtracking with pruning; exact runtime depends on implementation/pruning.

Space: $O(N^2)$ if the full board is stored, plus $O(N)$ recursion depth.

Quick Revision Table

Topic	Main Use	Time	Space
LCS	Longest common subsequence	$O(nm)$	$O(nm) / O(\min(n,m))$
LIS	Longest increasing subsequence	$O(n \log n)$	$O(n)$
Connected Components	Separate graph groups	$O(V+E)$	$O(V+E)$
Floyd-Warshall	All-pairs shortest paths	$O(V^3)$	$O(V^2)$
Segment Tree	Range query + update	Build $O(n)$; Q/U $O(\log n)$	$O(n)$
Kadane	Max contiguous sum	$O(n)$	$O(1)$
DSU Cycle Detection	Undirected cycle detection	$O(E \cdot \alpha(V)) \approx O(E)$	$O(V)$
Kruskal + DSU	Minimum Spanning Tree	$O(E \log E)$	$O(V+E)$
0/1 Knapsack	Take/skip each item	$O(nW)$	$O(nW) / O(W)$
Fractional Knapsack	Divisible item selection	$O(n \log n)$	$O(n)$
N-Queen	Non-attacking queens	$\sim O(N!)$	$O(N^2)$

Key Complexity Memory Guide

LCS $\rightarrow O(nm)$ **LIS** $\rightarrow O(n \log n)$ **DFS/BFS** $\rightarrow O(V+E)$
Floyd-Warshall $\rightarrow O(V^3)$ **Segment Tree** $\rightarrow O(\log n)$ query/update **Kadane** $\rightarrow O(n)$
DSU $\rightarrow O(\alpha(n))$ per operation **Kruskal** $\rightarrow O(E \log E)$ **0/1 Knapsack** $\rightarrow O(nW)$
Fractional Knapsack $\rightarrow O(n \log n)$ **N-Queen** $\rightarrow \sim O(N!)$